

经在前人的系统中得以证实并广泛使用，它使代码编写真正实现工程化，将已证实的技术表述成设计模式，也会使新系统开发者更加容易理解其设计思路。每种设计模式都是对13.4.3小节中某一种或多种OOD原则的体现。

2. 设计模式的关键元素

设计模式包含模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式等基本要素，其中的关键元素包括以下4个方面：

(1) 模式名称。给模式取一个助记名，用一两个词语来描述模式待解决的问题、解决方案和使用效果，以便更好地理解模式并方便开发人员之间的交流。

(2) 问题。描述应该在何时使用模式，即在解决何种问题时可使用该模式。有时，在问题部分会包括使用模式必须满足的一系列先决条件。

(3) 解决方案。描述设计的组成成分、它们之间的相互关系及各自的职责和协作方式。模式就像一个模板，可应用于多种不同场合，所以解决方案并不描述一个特定而具体的设计或实现，而是提供一个问题的抽象描述和具有一般意义的元素组合（类或对象组合）。

(4) 效果。描述模式应用的效果以及使用模式时应权衡的问题，即模式的优缺点。没有一种解决方案是完美的，每种设计模式都具有自己的优点，但也存在一些缺陷，它们对于评价设计选择和理解使用模式的代价和好处具有重要意义。模式效果有助于选择合适的模式，它不仅包括时间和空间的权衡，还包括对系统的灵活性、可扩展性或可移植性的影响。

13.5.2 设计模式分类

根据目的和用途不同，设计模式可分为创建型（Creational）模式、结构型（Structural）模式和行为型（Behavioral）模式三种。创建型模式主要用于创建对象，结构型模式主要用于处理类或对象的组合，行为型模式主要用于描述类或对象的交互以及职责的分配。

根据处理范围不同，设计模式可分为类模式和对象模式。类模式处理类和子类之间的关系，这些关系通过继承建立，在编译时刻就被确定下来，属于静态关系。对象模式处理对象之间的关系，这些关系在运行时刻变化，更具动态性。

1. 创建型模式

创建型模式对类的实例化过程（即对象的创建过程）进行了抽象，能够使软件模块做到与对象的创建和组织无关。创建型模式隐藏了对象是如何被创建和组合在一起的，以达到使整个系统独立的目的。创建型模式包括工厂方法模式、抽象工厂模式、原型模式、单例模式和建造者模式等。

(1) 工厂方法（Factory Method）模式。工厂方法模式又称为虚拟构造器（Virtual Constructor）模式或多态模式，属于类的创建型模式。在工厂方法模式中，父类负责定义创建对象的公共接口，而子类则负责生成具体的对象，这样做的目的是将类的实例化操作延迟到子类中完成，即由子类来决定究竟应该实例化（创建）哪一个类。

(2) 抽象工厂（Abstract Factory）模式。抽象工厂模式又称为Kit模式，属于对象创建型模式。抽象工厂模式是所有形式的工厂模式中最为抽象和最具一般性的一种形态，它提供了一个

创建一系列相关或相互依赖对象的接口，而无须指定它们具体的类。在抽象工厂模式中，引入了产品等级结构和产品族的概念。产品等级结构是指抽象产品与具体产品所构成的继承层次关系；产品族是同一个工厂所生产的一系列产品，即位于不同产品等级结构且功能相关联的产品组成的家族。当抽象工厂模式退化到只有一个产品等级结构时，即变成了工厂方法模式。

(3) 原型 (Prototype) 模式。在系统开发过程中，有时候有些对象需要被频繁创建，原型模式通过给出一个原型对象来指明所要创建的对象类型，然后通过复制这个原型对象的办法，创建出更多同类型的对象。原型模式是一种对象创建型模式，用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象。原型模式又可分为两种：浅克隆和深克隆。浅克隆仅仅复制所考虑的对象，而不复制它所引用的对象，也就是其中的成员对象并不复制；深克隆除了对象本身被复制外，对象包含的引用也被复制，即成员对象也被复制。

(4) 单例 (Singleton) 模式。单例模式确保某一个类只有一个实例，而且自行实例化并向整个系统提供这个实例，这个类称为单例类，它提供全局访问的方法。

(5) 建造者 (Builder) 模式。建造者模式强调将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。建造者模式是一步一步地创建一个复杂的对象，它允许用户只通过指定复杂对象的类型和内容就可以构建它们，用户不需要知道内部的具体构建细节。建造者模式属于对象创建型模式。

2. 结构型模式

结构型模式描述如何将类或对象结合在一起形成更大的结构。结构型模式描述两种不同的事物，即类与类的实例（对象），据此可以将结构型模式分为类结构型模式和对象结构型模式。结构型模式包括适配器 (Adapter) 模式、桥接 (Bridge) 模式、组合 (Composite) 模式、装饰 (Decorator) 模式、外观 (Facade) 模式、享元 (Flyweight) 模式和代理 (Proxy) 模式等。

(1) 适配器模式。适配器模式将一个接口转换成客户希望的另一个接口，从而使接口不兼容的那些类可以一起工作。适配器模式既可以作为类结构型模式，也可以作为对象结构型模式。在类适配器模式中，通过使用一个具体类将适配器适配到目标接口中；在对象适配器模式中，一个适配器可以将多个不同的适配者适配到同一个目标。

(2) 桥接模式。桥接模式将抽象部分与它的实现部分分离，使它们都可以独立地变化。它是一种对象结构型模式，又称为柄体 (Handle and Body) 模式或接口 (Interface) 模式。桥接模式类似于多重继承方案，但是多重继承方案往往违背了类的单一职责原则，其复用性比较差，桥接模式是比多重继承方案更好的解决方法。

(3) 组合模式。组合模式又称为整体 - 部分 (Part-Whole) 模式，属于对象的结构模式。在组合模式中，通过组合多个对象形成树形结构以表示整体 - 部分的结构层次。组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性。

(4) 装饰模式。装饰模式是一种对象结构型模式，可动态地给一个对象增加一些额外的职责，就增加对象功能来说，装饰模式比生成子类实现更为灵活。通过装饰模式，可以在不影响其他对象的情况下，以动态、透明的方式给单个对象添加职责；当需要动态地给一个对象增加功能，这些功能可以再动态地被撤销时，可使用装饰模式；当不能采用生成子类的方法进行扩

充时也可以使用装饰模式。

(5) 外观模式。外观模式是对象的结构模式，要求外部与一个子系统的通信必须通过一个统一的外观对象进行，为子系统中的一组接口提供一个一致的界面，外观模式定义了一个高层接口，这个接口使得这一子系统更加容易使用。

(6) 享元模式。享元模式是一种对象结构型模式，通过运用共享技术，有效地支持大量细粒度的对象。系统只使用少量的对象，而这些对象都很相似，状态变化很小，对象使用次数增多。享元对象能做到共享的关键是区分内部状态和外部状态。内部状态存储在享元对象内部并且不会随环境改变而改变，因此内部状态可以共享；外部状态是随环境改变而改变的、不可以共享的状态，享元对象的外部状态必须由客户端保存，并在享元对象被创建之后，在需要使用的时候再传入到享元对象内部，外部状态之间是相互独立的。

(7) 代理模式。代理模式是一种对象结构型模式，可为某个对象提供一个代理，并由代理对象控制对原对象的引用。代理模式能够协调调用者和被调用者，能够在一定程度上降低系统的耦合度，其缺点是请求的处理速度会变慢，并且实现代理模式需要额外的工作。

3. 行为型模式

行为型模式是对在不同的对象之间划分责任和算法的抽象化，它不仅仅是关于类和对象的，而且是关于它们之间的相互作用的。行为型模式分为类行为模式和对象行为模式两种，其中类行为模式使用继承关系在几个类之间分配行为，而对象行为模式则使用对象的聚合来分配行为。行为型模式包括职责链（Chain of Responsibility）模式、命令（Command）模式、解释器（Interpreter）模式、迭代器（Iterator）模式、中介者（Mediator）模式、备忘录（Memento）模式、观察者（Observer）模式、状态（State）模式、策略（Strategy）模式、模板方法（Template Method）模式、访问者（Visitor）模式等。

(1) 职责链模式。职责链模式是一种对象的行为型模式，避免请求发送者与接收者耦合在一起，让多个对象都有可能接收请求，将这些对象连接成一条链，并且沿着这条链传递请求，直到有对象处理它为止。职责链模式不保证每个请求都被接受，如果有一个请求没有明确的接收者，那么就不能保证它一定会被处理。

(2) 命令模式。命令模式是一种对象的行为型模式，类似于传统程序设计方法中的回调机制，它将一个请求封装为一个对象，从而可用不同的请求对客户进行参数化；对请求排队或者记录请求日志，以及支持可撤销的操作。命令模式是对命令的封装，将发出命令的责任和执行命令的责任分割开，委派给不同的对象，以实现发送者和接收者完全解耦，提供更大的灵活性和可扩展性。

(3) 解释器模式。解释器模式属于类的行为型模式，描述了如何为语言定义一个文法，如何在该语言中表示一个句子，以及如何解释这些句子，这里的“语言”是使用规定格式和语法的代码。解释器模式主要用在编译器中，在应用系统开发中很少用到。

(4) 迭代器模式。迭代器模式是一种对象的行为型模式，提供了一种方法来访问聚合对象，而不用暴露这个对象的内部表示。迭代器模式支持以不同的方式遍历一个聚合对象，复杂的聚合可用多种方法来进行遍历；允许在同一个聚合上可以有多个遍历，每个迭代器保持它自己的

遍历状态，因此，可以同时进行多个遍历操作。

(5) 中介者模式。中介者模式是一种对象的行为型模式，通过一个中介对象来封装一系列的对象交互。中介者使得各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互。中介者对象的存在保证了对象结构上的稳定，也就是说，系统的结构不会因为新对象的引入带来大量的修改工作。

(6) 备忘录模式。备忘录模式确保在不破坏封装的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，这样可以在以后将对象恢复到原先保存的状态。备忘录模式提供了一种状态恢复的实现机制，使得用户可以方便地回到一个特定的历史步骤。

(7) 观察者模式。观察者模式又称为发布 - 订阅模式、模型 - 视图模式、源 - 监听器模式或从属者 (Dependents) 模式，是一种对象的行为型模式。它定义了对象之间的一种一对多的依赖关系，使得每当一个对象状态发生改变时，其相关依赖对象都得到通知并被自动更新。观察者模式的优点在于实现了表示层和数据层的分离，并定义了稳定的更新消息传递机制，类别清晰，抽象了更新接口，使得相同的数据层可以有各种不同的表示层。

(8) 状态模式。状态模式是一种对象的行为型模式，允许一个对象在其内部状态改变时改变它的行为，对象看起来似乎修改了它的类。状态模式封装了状态的转换过程，但是它需要枚举可能的状态，因此，需要事先确定状态种类，这也导致在状态模式中增加新的状态类时将违反开闭原则，新的状态类的引入将需要修改与之能够进行转换的其他状态类的代码。状态模式的使用必然会增加系统类和对象的个数。

(9) 策略模式。策略模式是一种对象的行为型模式，定义一系列算法，并将每一个算法封装起来，并让它们可以相互替换。策略模式让算法独立于使用它的客户而变化，其目的是将行为和环境分隔，当出现新的行为时，只需要实现新的策略类。

(10) 模板方法模式。模板方法模式是一种类的行为型模式，用于定义一个操作中算法的骨架，而将一些步骤延迟到子类中。模板方法模式使得子类可以不改变一个算法的结构即可重定义该算法的某些特定步骤，其缺点是对于不同的实现，都需要定义一个子类，这会导致类的个数增加，但是更加符合类职责的分配原则，使得类的内聚性得以提高。

(11) 访问者模式。访问者模式是一种对象的行为型模式，用于表示一个作用于某对象结构中的各元素的操作，它使得用户可以在不改变各元素的类的前提下定义作用于这些元素的新操作。访问者模式使得增加新的操作变得很容易，但在一定程度上破坏了封装性。

13.6 输入 / 输出原型设计

系统输入 / 输出设计是与用户息息相关的，输入 / 输出设计对于系统的开发人员以及用户极为重要。系统输入 / 输出设计可以帮助开发人员更好地理解系统的需求和功能，从而确保系统的正确性和稳定性；可以使系统的代码更加清晰和易于维护，从而降低维护成本和风险；使系统更加灵活和可扩展，从而满足不同用户的需求和变化的市场需求；使系统更加易于使用和友好，从而提高用户的满意度和忠诚度；可以使开发人员更加高效地开发系统，从而降低开发成本和时间。